



本小节内容

指针与动态内存申请

栈空间与堆空间的差异（了解）

1 指针与动态内存申请

很多读者在学习 C 语言的数组后都会觉得数组长度固定很不方便，其实 C 语言的数组长度固定是因为其定义的**整型、浮点型、字符型变量、数组变量**都在栈空间中，而栈空间的大小在编译时是确定的。如果使用的空间大小不确定，那么就要使用**堆空间**。请看下面例子

【例 1.1】动态内存申请。

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    int i;
    char *p;
    scanf("%d",&i); //输入要申请的空间大小
    p=(char*)malloc(i); //使用 malloc 动态申请堆空间
    strcpy(p,"malloc success");
    puts(p);
    free(p); //free 时必须使用 malloc 申请时返回的指针值，不能进行任何偏移
    printf("free success\n");
    return 0;
}
```

首先我们来看 malloc 函数。在执行 `#include <stdlib.h> void *malloc(size_t size);` 时，需要给 malloc 传递的参数是一个整型变量，因为这里的 size_t 即为 int；返回值为 void* 类型的指针，void* 类型的指针只能用来存储一个地址而不能进行偏移，因为 malloc 并不知道我们申请的空间用来存放什么类型的数据，所以确定要用来存储什么类型的数据后，都会将 void* 强制转换为对应的类型。在例 1.1 中我们用来存储字符，所以将其强制转换为 char* 类型。

同时需要注意指针本身大小，和其指向的空间大小，是两码事，不能和前面的变量类比去理解！

如下图所示，定义的整型变量 i、指针变量 p 均在 main 函数的栈空间中，通过 malloc 申请的空间会返回一个堆空间的首地址，我们把首地址存入变量 p。知道了首地址，就可以通过 strcpy 函数往对应的空间存储字符数据。

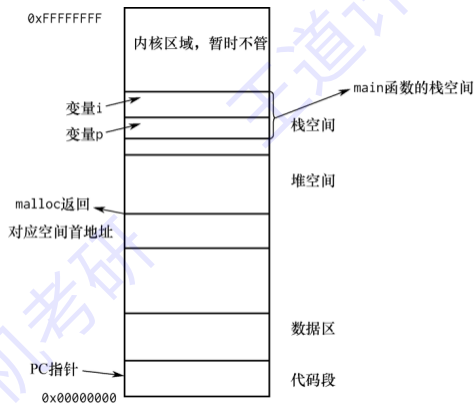


图 1.1 栈空间与堆空间

既然都是内存空间，为什么还要分栈空间和堆空间呢？栈是计算机系统提供的数据结构，计算机会在底层对栈提供支持：分配专门的寄存器存放栈的地址，压栈操作、出栈操作都有专门的指令执行，这就决定了栈的效率比较高；堆则是 C/C++ 函数库提供的数据结构，它的机制很复杂，例如为了分配一块内存，库函数会按照一定的算法（具体的算法请参考关于数据结构、操作系统的书籍）在堆内存中搜索可用的足够大小的空间，如果没有足够大小的空间（可能由于内存碎片太多），那么就有可能调用系统功能去增加程序数据段的内存空间，这样就有机会分到足够大小的内存，然后返回。显然，堆的效率要比栈低得多。（这段了解即可）

栈空间由系统自动管理，而堆空间的申请和释放需要自行管理，所以在具体例子中需要通过 free 函数释放堆空间。free 函数的头文件及格式为

```
#include <stdlib.h>
void free(void *ptr);
```

其传入的参数为 void 类型指针，任何指针均可自动转为 void* 类型指针，所以我们把 p 传递给 free 函数时，不需要强制类型转换。p 的地址值必须是 malloc 当时返回的地址值，不能进行偏移，也就是在 malloc 和 free 之间不能进行 p=p+1 等改变变量 p 的操作，原因是申请一段堆内存空间时，内核帮我们记录的是起始地址和大小，所以释放时内核用对应的首地址进行匹配，匹配不上时，进程就会崩溃。如果要偏移进而存储数据，那么可以定义两个指针变量来解决。

接下来我们来试试先偏移 p 再执行 free，看看程序崩溃的效果。

下面我们了解栈空间和堆空间在时间有效性上的差异。

2 栈空间与堆空间的差异

【例 2.1】栈空间与堆空间的差异。（记住即可）

```
#include <stdio.h>
```

```
#include <stdlib.h>
```



```
#include <string.h>
```

```
//函数栈空间释放后，函数内的所有局部变量消失
```

```
char* print_stack()
```

```
{
```

```
    char c[17]="I am print_stack";
```

```
    puts(c);
```

```
    return c;
```

```
}
```

```
//堆空间不会因函数执行结束而释放
```

```
char* print_malloc()
```

```
{
```

```
    char *p;
```

```
    p=(char*)malloc(20);
```

```
    strcpy(p,"I am print_malloc");
```

```
    puts(p);
```

```
    return p;
```

```
}
```

```
int main()
```

```
{
```

```
    char *p;
```



```
p=print_stack(); //数据放在栈空间

printf("p=%s\n",p);

p=print_malloc(); //数据放在堆空间

puts(p);

return 0;

}
```

例 2.1 中代码的执行结果如下图所示。为什么第二次打印会有异常？原因是 `print_stack()` 函数中的字符串存放在栈空间中，函数执行结束后，栈空间会被释放，字符数组 `c` 的原有空间已被分配给其他函数使用，因此在调用 `print_stack()` 函数后，`printf("p=%s\n",p);` 中的 `p` 不能获取栈空间的数据。而 `print_malloc()` 函数中的字符串存放在堆空间中，堆空间只有在执行 `free` 操作后才会释放，否则在进程执行过程中会一直有效。

```
I am print_stack
p=(null)
I am print_malloc
I am print_malloc
```

图 2.1 例 2.1 中代码的执行结果